

```
# =====  
# CORNER DETECTION FOR OBJECT TRACKING  
# Shi-Tomasi + Lucas-Kanade Optical Flow  
# =====  
  
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
from google.colab import files  
  
# -----  
# 1. Upload Video  
# -----  
uploaded = files.upload()  
video_name = list(uploaded.keys())[0]  
  
cap = cv2.VideoCapture(video_name)  
  
# -----  
# 2. Parameters for Corner Detection  
# -----  
feature_params = dict(  
    maxCorners=100,  
    qualityLevel=0.3,  
    minDistance=7,  
    blockSize=7  
)  
  
# Parameters for Lucas-Kanade Optical Flow  
lk_params = dict(  
    winSize=(15, 15),  
    maxLevel=2,  
    criteria=(  
        cv2.TERM_CRITERIA_EPS |  
        cv2.TERM_CRITERIA_COUNT,  
        10.
```

```
        0.03
    )
)

# -----
# 3. Read First Frame
# -----
ret, old_frame = cap.read()

if not ret:
    print("Error: Video could not be read.")
    cap.release()
else:
    old_gray = cv2.cvtColor(
        old_frame,
        cv2.COLOR_BGR2GRAY
    )

    # Detect corners in first frame
    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        mask=None,
        **feature_params
    )

    # Mask for drawing tracks
    mask = np.zeros_like(old_frame)

    # Store tracking results
    tracking_frames = []

    # -----
    # 4. Track Corners Across Frames
    # -----
    while True:

        ret, frame = cap.read()
```

```
if not ret:
    break

frame_gray = cv2.cvtColor(
    frame,
    cv2.COLOR_BGR2GRAY
)

# Calculate optical flow
p1, status, error = cv2.calcOpticalFlowPyrLK(
    old_gray,
    frame_gray,
    p0,
    None,
    **lk_params
)

if p1 is None:
    break

# Select successfully tracked points
good_new = p1[status == 1]
good_old = p0[status == 1]

# -----
# 5. Draw Tracking Lines
# -----
for new, old in zip(good_new, good_old):

    a, b = new.ravel()
    c, d = old.ravel()

    a, b = int(a), int(b)
    c, d = int(c), int(d)

    # Draw motion path
    mask = cv2.line(
```

```

        mask,
        (a, b),
        (c, d),
        (0, 255, 0),
        2
    )

    # Draw tracked corner
    frame = cv2.circle(
        frame,
        (a, b),
        5,
        (0, 0, 255),
        -1
    )

    # Combine tracking lines and frame
    output = cv2.add(
        frame,
        mask
    )

    tracking_frames.append(
        cv2.cvtColor(
            output,
            cv2.COLOR_BGR2RGB
        )
    )

    # Update previous frame and points
    old_gray = frame_gray.copy()

    p0 = good_new.reshape(
        -1, 1, 2
    )

    # -----
    # 6. Re-detect Corners if Needed

```

```
# -----  
if len(p0) < 10:  
  
    p0 = cv2.goodFeaturesToTrack(  
        old_gray,  
        mask=None,  
        **feature_params  
    )  
  
cap.release()  
  
# -----  
# 7. Display Sample Tracking Frames  
# -----  
if len(tracking_frames) > 0:  
  
    indexes = np.linspace(  
        0,  
        len(tracking_frames) - 1,  
        min(6, len(tracking_frames)),  
        dtype=int  
    )  
  
    plt.figure(figsize=(15, 8))  
  
    for i, idx in enumerate(indexes):  
  
        plt.subplot(2, 3, i + 1)  
  
        plt.imshow(  
            tracking_frames[idx]  
        )  
  
        plt.title(  
            "Frame " + str(idx + 1)  
        )  
  
        plt.axis("off")
```

```
plt.tight_layout()
plt.show()

print("Corner-based tracking completed.")
print(
    "Total frames processed:",
    len(tracking_frames)
)
```

[Choose Files](#) Screenshot... 084833.png

Screenshot 2026-05-09 084833.png(image/png) - 676711 bytes, last modified: 5/9/2026 - 100% done

Saving Screenshot 2026-05-09 084833.png to Screenshot 2026-05-09 084833 (2).png

Corner-based tracking completed.

Total frames processed: 0

